CSCI426 - ADVANCED WEB PROGRAMMING

# EMBER

*Worth the craving.*

## Restaurant Ordering Website

Project Phase 1 - Project Report

| | |
|---|---|
| Student | Jaafar Sabra |
| Student ID | 72230515 |
| Instructor | Ghadir Khalil |
| University | Lebanese International University (LIU) |
| Project mode | Individual |
| Date | 8 November 2026 |

GitHub: github.com/JaafarSabra/EMBER-Restaurant

Website: ember-restaurant-rust.vercel.app

01

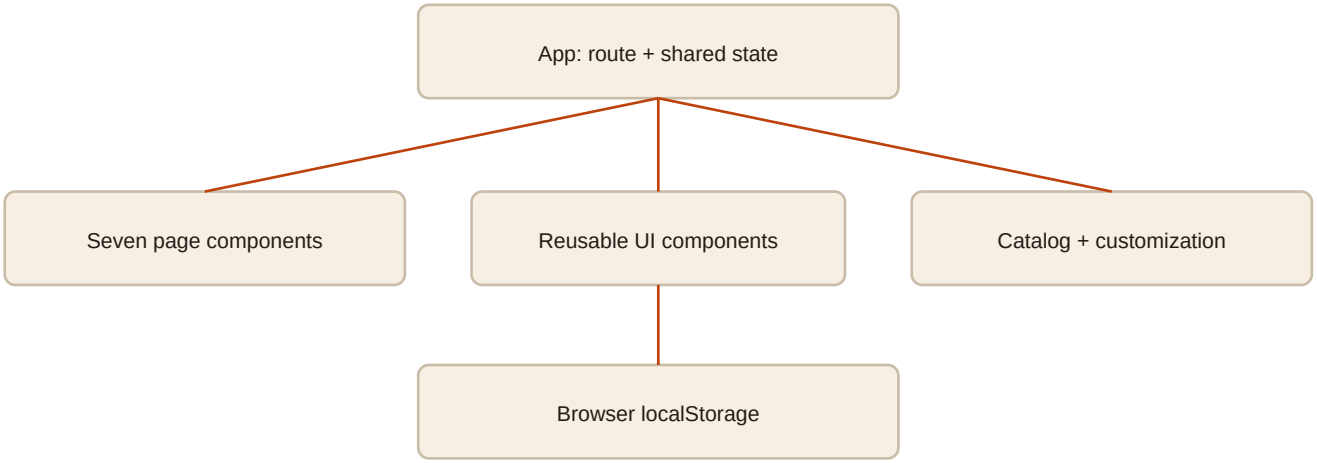
# *Abstract*

EMBER is a responsive restaurant ordering website built with React. It allows customers to browse a varied menu, search and filter dishes, view product details, customize items, manage a shopping bag, choose delivery or pickup, and view order progress. Its interface combines a charcoal header, ivory backgrounds, orange accents and food photography.

The website includes Home, Our Story, Experience, Menu, Contact, Checkout and Tracking pages. React state updates selections, quantities and calculated totals. Browser localStorage retains the bag and latest order. Order tracking is simulated, and the contact form saves messages locally. The application does not transmit real orders, payments or email.

02

# System design




| Component / module | Responsibility |
| --- | --- |
| src/App.jsx | Route selection, shared cart state, customization dialogs and order creation. |
| src/pages/ | Home, Our Story, Experience, Menu, Contact, Checkout and Tracking. |
| src/components/UI.jsx | Reusable controls, product cards, photo viewer, modal behavior and map. |
| src/data/menu.js | Catalog, product images and price formatting. |
| src/data/customization.js | Dish-specific choices, extras and prices. |
| style.css / public/assets/ | Responsive styling, motion and local images. |

## Ordering flow

Menu → customization → bag → checkout → tracking. Each selected option contributes to the unit price. Line total equals unit price multiplied by quantity. Delivery adds $3; pickup has no delivery fee. Checkout validates the required fields, saves an order snapshot, and clears the bag.

## Navigation and persistence

Hash navigation selects page components. React state updates the visible interface without full page reloads. localStorage keys ember-cart, ember-order and ember-contact retain data within the current browser. Responsive CSS adapts layouts and navigation to desktop and mobile screens.

03

# Technologies used

| Technology | Use |
| --- | --- |
| React 19 / JavaScript JSX | Reusable components, event handlers, hooks and application state. |
| Vite 6 | Local development server and optimized production build. |
| HTML5 / CSS | Semantic content, responsive layouts, typography and transitions. |
| Browser localStorage | Local persistence of cart, latest order and contact message. |
| Git / GitHub | Version control and source repository. |
| Vercel | Frontend deployment. |
| Google Maps / Google Fonts | Beirut map and interface fonts. |

## Design and asset sources

The EMBER flame wordmark and vector icons are authored in source. Food and editorial images combine OpenAI-generated imagery and Unsplash photographs served locally. Fonts are Barlow Condensed, DM Sans and Libre Caslon Display. AI assistance was used for design, implementation and documentation. No pre-built website template was used.

04

# *Code snippets - key parts only*

## Menu filtering - src/pages/Menu.jsx

The selected category and search query are applied together.

```
filtered = menu.filter(
  (x) =>
    (cat === "All" || x.category === cat) &&
    (x.name + " " + x.description)
      .toLowerCase()
      .includes(search.trim().toLowerCase()),
);
```

## Customization pricing - src/App.jsx

Selected options and extras contribute to the product price.

```
const extraPrice = config
  ? config.groups.reduce(
      (sum, g) => sum +
        (g.options.find(o => o.label === choices[g.name])?.price || 0),
      0,
    ) + config.extras
      .filter(x => extras.includes(x.label))
      .reduce((sum, x) => sum + x.price, 0)
  : 0;
```

04 / CONTINUED

# *Cart updates and persistence*

## src/pages/Checkout.jsx

Quantity changes update the matching cart item, cap quantity at 99, and remove an item when its quantity reaches zero.

```
const change = (key, n) =>
    setCart((c) =>
      c
        .map((x) =>
          x.key === key ? { ...x, qty: Math.min(99, x.qty + n) } : x,
        )
        .filter((x) => x.qty > 0),
    );
```

## src/App.jsx

Whenever the cart changes, an effect serializes it to localStorage. The initial cart state reads the saved ember-cart value.

```
useEffect(() => {
    localStorage.setItem("ember-cart", JSON.stringify(cart));
  }, [cart]);
```

04 / CONTINUED

# Checkout validation and order creation

## src/pages/Checkout.jsx - selected form excerpts

Native HTML validation requires a name and phone number. The phone pattern accepts 7-20 permitted characters; the address field is required when Delivery is selected. The valid form submits its values to placeOrder.

```
<input name="name" required autoComplete="name"
  placeholder="Enter your full name" />

// Phone field validation attributes:
required
pattern="[+0-9() .-]{7,20}"

// Form submit handler:
onSubmit={(e) => {
  e.preventDefault();
  placeOrder(Object.fromEntries(new FormData(e.target)));
}}
```

## src/App.jsx

Order creation calculates totals, stores the current items and customer details, clears the bag, and opens Tracking.

```
const placeOrder = (data) => {
    const subtotal = cart.reduce((a, x) => a + x.qty * x.unitPrice, 0),
      fee = mode === "Delivery" ? 3 : 0,
      newOrder = {
        ...data,
        id: "EM" + String(Date.now()).slice(-6),
        time: Date.now(),
        mode,
        subtotal,
        fee,
        total: subtotal + fee,
        items: cart,
      };
    localStorage.setItem("ember-order", JSON.stringify(newOrder));
    setOrder(newOrder);
    setCart([]);
    location.hash = "tracking";
  };
```